

WEEK 3

AI Tools Assignment: AI Tools and Applications

1. Short Answer Questions

Q1: Explain the primary differences between TensorFlow and PyTorch. When would you choose one over the other?

A:

TensorFlow and PyTorch are both deep learning frameworks, but they differ in their design philosophies.

- **TensorFlow** emphasizes production readiness and scalability. It has strong ecosystem tools like TensorBoard for visualization and TensorFlow Serving for deployment.
- **PyTorch** is more developer-friendly and dynamic, making it great for research and experimentation.
You'd typically choose **PyTorch** for rapid prototyping and academic projects, while **TensorFlow** suits **enterprise-scale applications** and deployment pipelines.

Q2: Describe two use cases for Jupyter Notebooks in AI development.

A:

1. **Exploratory Data Analysis (EDA):** Researchers use notebooks to visualize data, test hypotheses, and perform data cleaning interactively.
2. **Model Development & Experiment Tracking:** Developers can build, test, and tune machine learning models cell by cell, recording results inline for easy comparison and documentation.

Q3: How does spaCy enhance NLP tasks compared to basic Python string operations?

A:

spaCy provides **linguistic structure** (like tokenization, part-of-speech tagging, named entity recognition, and dependency parsing) that raw string operations lack.

Instead of manually splitting and searching text, spaCy offers pre-trained language models that understand grammar, entities, and context — making NLP tasks more accurate and efficient.

2. Comparative analysis

Feature	Scikit-learn	TensorFlow
Target Applications	Classical ML (regression, clustering, classification)	Deep Learning (neural networks, image and text models)
Ease of Use	Very beginner-friendly with simple APIs	Steeper learning curve, more setup needed
Community Support	Excellent for traditional ML, huge library base	Very active deep learning community and Google support

3. Output Screenshots

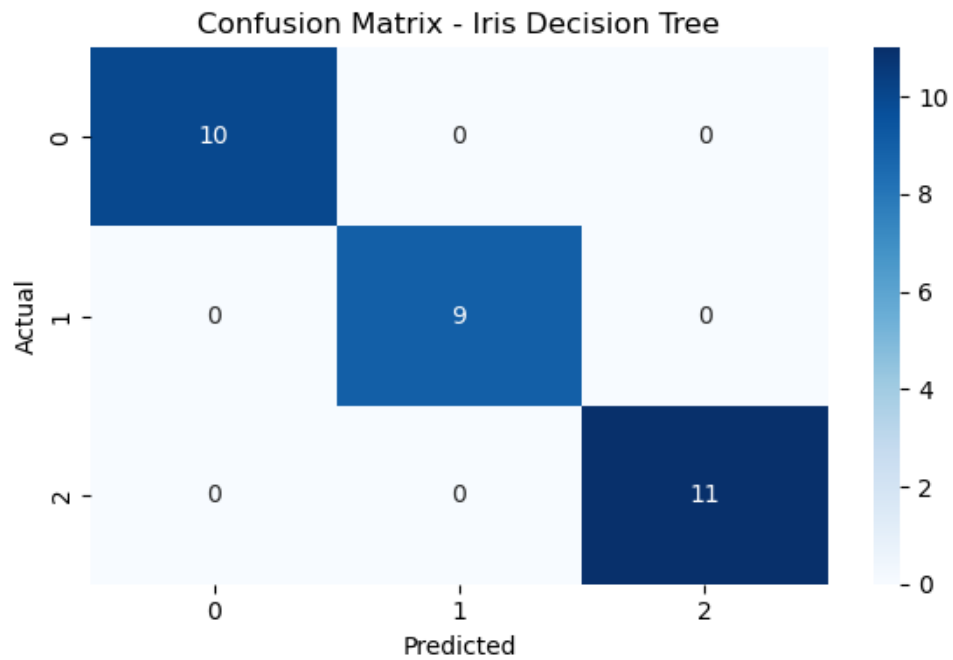


Figure 1...Confusion Matrix

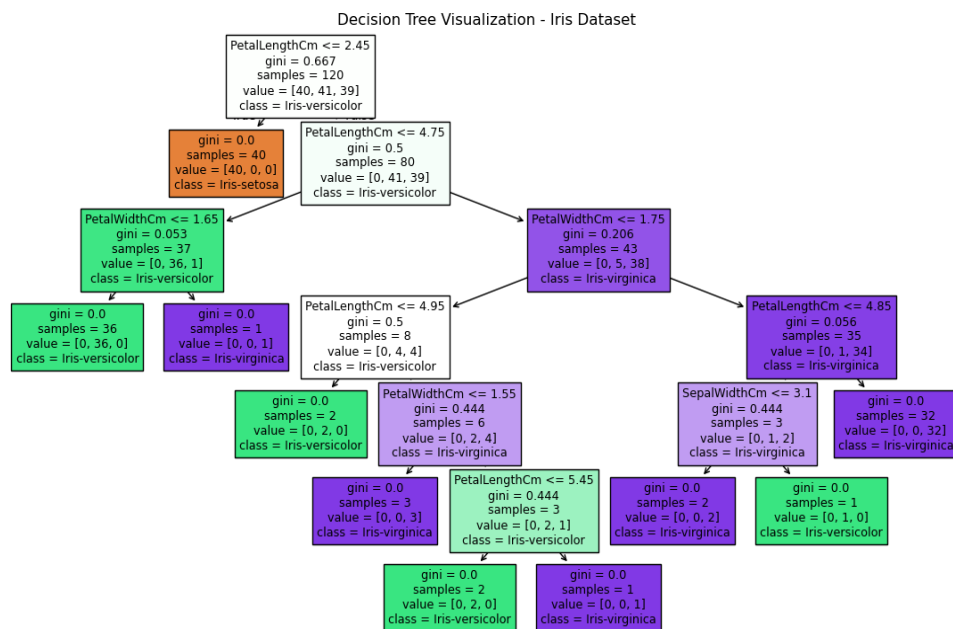


Figure 2...Decision Tree Visualization

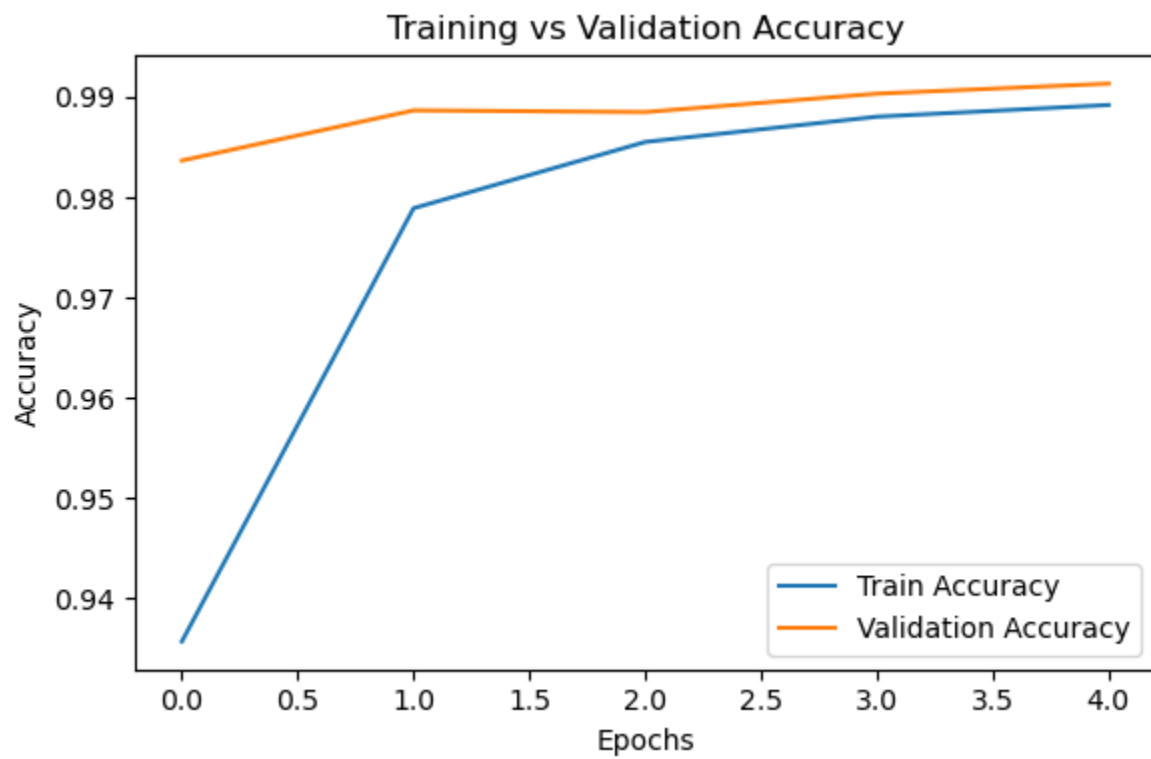


Figure 3...mnist_accuracy_graph

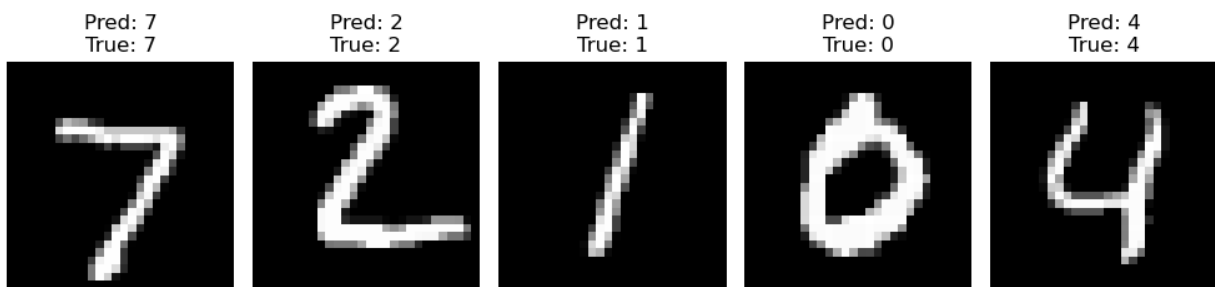


Figure 4...Mnist sample predictions

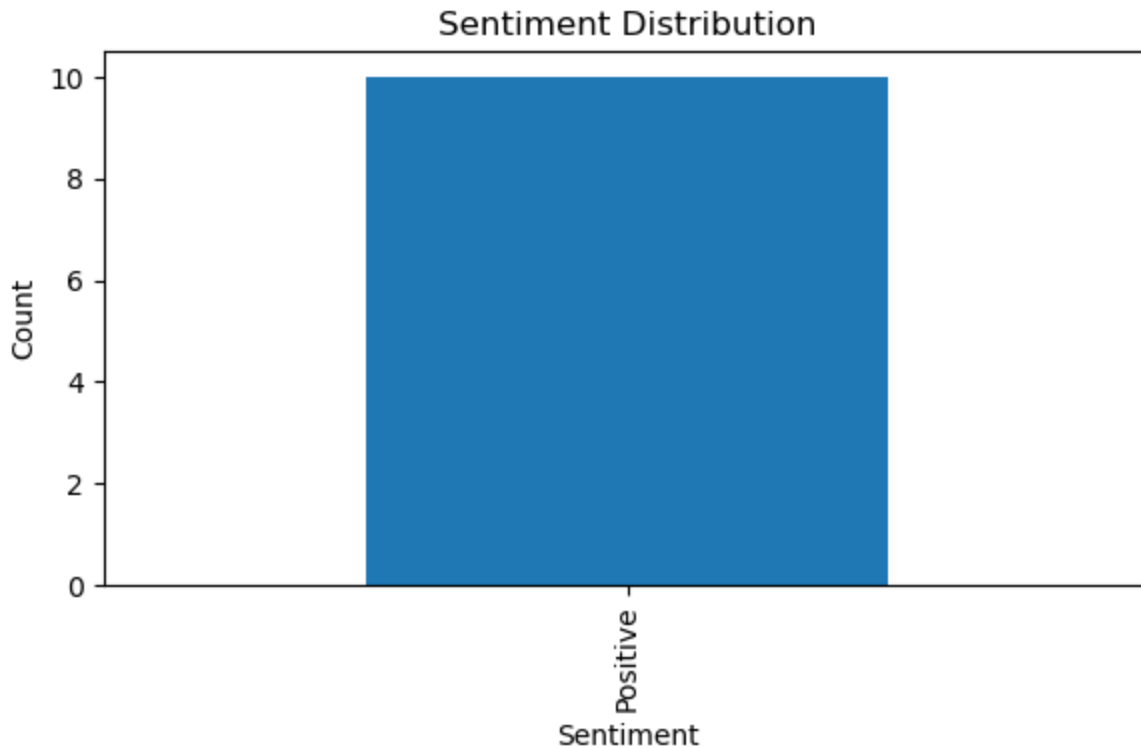


Figure 5...Spacy ner Output

Part 3 — Ethics and Optimization

1. Ethical Considerations

Potential biases

MNIST classifier

- **Sampling and demographic bias.** MNIST digits come mostly from particular sources (e.g., US school assignments / forms). Handwriting styles from other populations or ages may be underrepresented, so performance can drop on different handwriting styles.
- **Class imbalance or label noise.** If the training set is skewed (rare digits underrepresented) or labels are incorrect, model performance on those classes will be worse.
- **Input distribution shift.** Real-world inputs (camera photos, rotated / noisy digits) differ from clean 28×28 scans; model can be brittle.

Amazon Reviews NER / sentiment

- **Selection bias.** Reviews come from users who choose to post; extreme opinions (very positive or negative) are overrepresented.

- **Cultural and language bias.** Sentiment words and expressions vary across regions and dialects; a rule-based sentiment approach may mislabel nonstandard phrasing.
- **Brand / popularity bias.** Popular products have more reviews, giving the model more examples and better apparent performance; models may underperform on niche brands.
- **Annotator bias or label noise.** If sentiment labels are weak or inferred from star ratings, they may not reflect nuanced opinion.

Consequences

- Wrong business decisions, unfair treatment of products or customers, or failure to generalize to new user groups.

How to detect & measure bias

- **Slice-based evaluation.** Measure performance across meaningful subgroups (e.g., by product category, brand, review length, review language, handwriting style). Look for gaps in accuracy/precision/recall.
- **Confusion analysis.** Inspect which classes the model confuses more often in specific slices.

Mitigations and tools

Using TensorFlow Fairness Indicators

- **What it does.** Provides dashboards and slice-aware metrics so you can compare model performance across subgroups (slices) and visualise parity gaps.
- **How to use it:** export evaluation metrics (predictions, labels, and slice keys such as brand or category), then feed to Fairness Indicators to examine metrics per slice.
- **Actions after detection:** re-balance training data for underperforming slices; apply per-slice thresholds; collect more representative data; use reweighting or adversarial debiasing.

Using spaCy's rule-based systems

- **Entity Ruler & custom pipeline components.** Create deterministic patterns for product and brand names (especially when product names follow templates) to reduce NER false negatives/positives.
- **Normalization & canonicalization.** Map detected brand name variants to canonical forms (e.g., “Samsng” → “Samsung”) before downstream sentiment analysis to avoid brand-level bias.
- **Rule-based filters for noise.** Remove obvious spam or bot-like reviews (short, repeated phrases, or many links) using simple heuristics to improve data quality before training.
- **Human-in-the-loop corrections.** Use rule-based extraction for high-confidence cases and send low-confidence or ambiguous cases to human reviewers.

Other best practices

- Data augmentation (for MNIST): add rotations, translations, and noise to emulate varied handwriting.
- Cross-validation across different domains (time or geography).
- Provide uncertainty estimates and fallbacks (e.g., “unsure — escalate to human review”).
- Document dataset provenance and maintain a bias-risk register in the project README.

2. Troubleshooting Challenge — Debugging a buggy TensorFlow script

Below I present a short representative **buggy TensorFlow script** (common mistakes), then a **fixed version** with explanations. You can include this in your report as the debug evidence.

Buggy script (example)

```

1  # buggy_mnist.py
2  import tensorflow as tf
3  from tensorflow.keras import layers, models
4
5  # Load data
6  (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
7
8  # Normalize but missing channel dimension
9  x_train = x_train / 255.0
10 x_test = x_test / 255.0
11
12 # Build model (input shape mismatch risk)
13 model = models.Sequential([
14     layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28)), # BUG: missing channel
15     layers.MaxPooling2D(2,2),
16     layers.Flatten(),
17     layers.Dense(10, activation='softmax')
18 ])
19
20 # Wrong loss for one-hot vs sparse labels
21 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy']) # BUG if y are ints
22
23 model.fit(x_train, y_train, epochs=3, batch_size=64)
24

```

Common errors in this snippet

1. Conv2D expects a 3D input shape (height, width, channels). The code supplies (28,28) causing a shape mismatch.
2. Labels y_train are integers 0–9 (sparse). Using categorical_crossentropy requires one-hot labels; mismatch causes training to fail or give poor results.
3. No channel dimension added after normalizing, so model input will fail.

```

1  # fixed_mnist.py
2  import tensorflow as tf
3  from tensorflow.keras import layers, models
4
5  # Load data
6  (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
7
8  # Normalize and add channel dimension (Step A)
9  x_train = x_train.astype('float32') / 255.0
10 x_test = x_test.astype('float32') / 255.0
11 x_train = x_train[ ..., tf.newaxis] # shape becomes (N, 28, 28, 1)
12 x_test = x_test[ ..., tf.newaxis]
13
14 # Build model (input shape includes channels)
15 model = models.Sequential([
16     layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
17     layers.MaxPooling2D((2,2)),
18     layers.Conv2D(64, (3,3), activation='relu'),
19     layers.MaxPooling2D((2,2)),
20     layers.Flatten(),
21     layers.Dense(128, activation='relu'),
22     layers.Dropout(0.3),
23     layers.Dense(10, activation='softmax')
24 ])
25
26 # Use sparse_categorical_crossentropy when labels are integers (Step B)
27 model.compile(optimizer='adam',
28               loss='sparse_categorical_crossentropy',
29               metrics=['accuracy'])
30
31 # Fit (add validation split for monitoring)
32 history = model.fit(x_train, y_train, epochs=5, batch_size=64, validation_split=0.1)
33
34 # Evaluate
35 test_loss, test_acc = model.evaluate(x_test, y_test)
36 print("Test accuracy:", test_acc)
37

```

Fix explanations

- **Step A:** added `[..., tf.newaxis]` to include the channels dimension so Conv2D receives (28,28,1).
- **Step B:** changed loss to `sparse_categorical_crossentropy` because `y_train` are integer labels. Alternatively, one could convert labels to one-hot and keep `categorical_crossentropy`.
- Also added a second Conv2D and dropout for better performance and a `validation_split` for monitoring.

Other debugging tips

- Always print `x_train.shape` and `y_train.shape` before building the model to confirm shapes.

- If using `from_logits=True` in `losses.SparseCategoricalCrossentropy`, make sure the model's final activation is linear (no softmax), or set `from_logits=False` with softmax.
- If training is unstable, reduce learning rate or add gradient clipping.